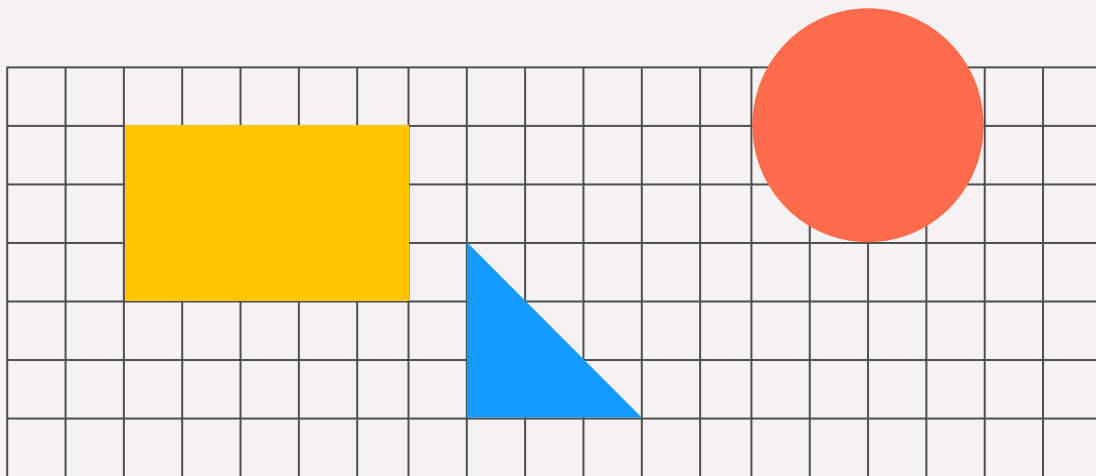


Spec-to-Ship @ IBM Bob hackathon walkthrough

Chi L. Nguyen
(github username: lchinguyen)
LinkedIn: Chi Nguyen



Spec-to-Ship

Converts plain-English feature requests into structured engineering tasks and automatically opens GitHub pull requests.

The project demonstrates how IBM Bob, watsonx Orchestrate, and the GitHub API work together to accelerate development workflows

- Developer reviews the generated PR instead of starting from scratch!

Plain-English Feature Intake:

Users submit natural-language software requirements through a React frontend interface

Example:

```
> Add JWT login authentication with protected routes.
```



2. watsonx Orchestrate uses

- Spec Parser Agent for task decomposition

- PR Description Agent for pull request generation

3. IBM Bob IDE analyze repo

- backend pipeline analysis

- production-readiness improvements

- README/documentation generation

4. GitHub API creates a branch, commits generated files, and opens a pull request.

Demo Workflow

Architecture

Frontend

User submits feature request



Backend

Node.js/Express pipeline processes request



watsonx Orchestrate:

Spec Parser Agent: for task decomposition
PR Description Agent: for pull request generation



GitHub Automation:

Create branch
Commit generated files
Open pull request



Demo repo `spec-to-ship-demo-target` Generated Pull Request

Frontend runs here: <http://localhost:5173>

Backend runs here: <http://localhost:8080>

The frontend sends requests to a Node.js/Express backend pipeline API that creates processing jobs and manages the engineering workflow. The frontend demo interacts directly with this backend workflow.

Testing repository where PRs were automatically generated show in video demo

Environment setup in .env : IBM Resources + Github REST API

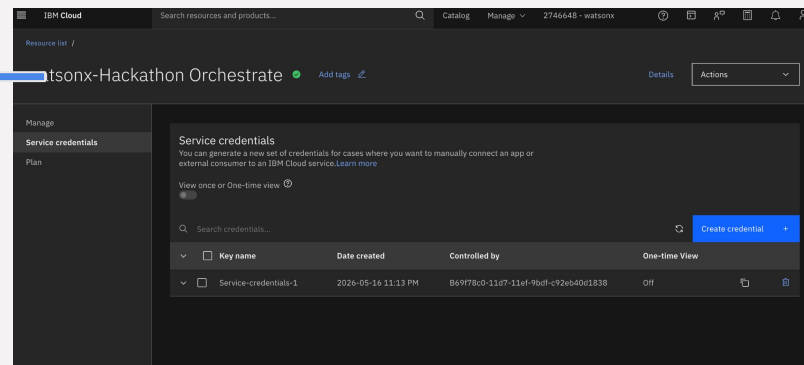
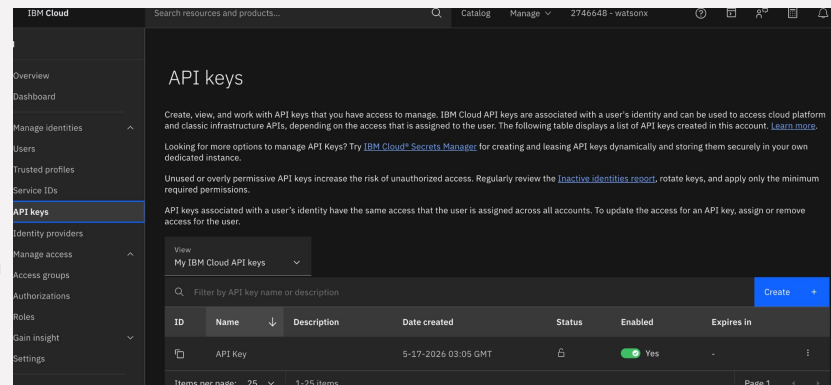
IBM Cloud API Key :
authenticate into IBM Cloud services,
access watsonx Orchestrate APIs,
authorize backend communication with IBM AI resources

```
spec-to-ship / backend / .env.example

Ichinguyen feat: complete spec-to-ship hackathon project

Code Blame 17 lines (14 loc) · 356 Bytes

1 PORT=8080
2 FRONTEND_URL=http://localhost:5173
3
4 WATSONX_API_KEY=
5 WATSONX_URL=https://us-south.ml.cloud.ibm.com
6 WATSONX_PROJECT_ID=
7 WATSONX_MODEL_ID=ibm/granite-34b-code-instruct
8
9 ORCHESTRATE_URL=
10 ORCHESTRATE_INSTANCE_ID=
11 ORCHESTRATE_AGENT_ID=
12 ORCHESTRATE_PR_AGENT_ID=
13
14 GITHUB_TOKEN=
15 GITHUB_OWNER=
16 GITHUB_REPO=spec-to-ship-demo-target
17 GITHUB_BASE_BRANCH=main
```



Personal access tokens (classic)

Tokens you have generated that can be used to access the [GitHub API](#).

spec-to-ship-hackathon — repo, workflow

⚠ This token has no expiration date.

GitHub Personal Access
Token used by Octokit
to:

- create branches,
- commit files,
- open PRs.

watsonx Orchestrate Service Credentials:

- Orchestrate instance credentials,
- instance URL,
- instance ID,
- service API connection info.

IBM watsonx Orchestrate Agents

The screenshot shows the IBM watsonx Orchestrate interface for the 'Spec Parser Agent'. The top navigation bar includes 'Agent chat / Manage agents / Spec Parser Agent'. The left sidebar has tabs for 'Profile', 'Knowledge', 'Toolset', 'Behavior' (selected), and 'Channels'. The main content area is divided into two sections: 'Behavior' and 'Guidelines'. The 'Behavior' section contains instructions for the agent, such as 'Do not use fields named task, steps, dependencies, outputArtifacts, or estimatedTimeHours.' and 'Do not return markdow.'. The 'Guidelines' section has a button to 'Add Guideline'. On the right, there is a chat window titled 'Running in debug mode' showing a message from 'Spec Parser Agent' at 4:40 AM, which includes a JSON object with fields like 'featureName', 'summary', and 'tasks'.

Converts feature specifications into structured JSON engineering task plans.

Example output includes:

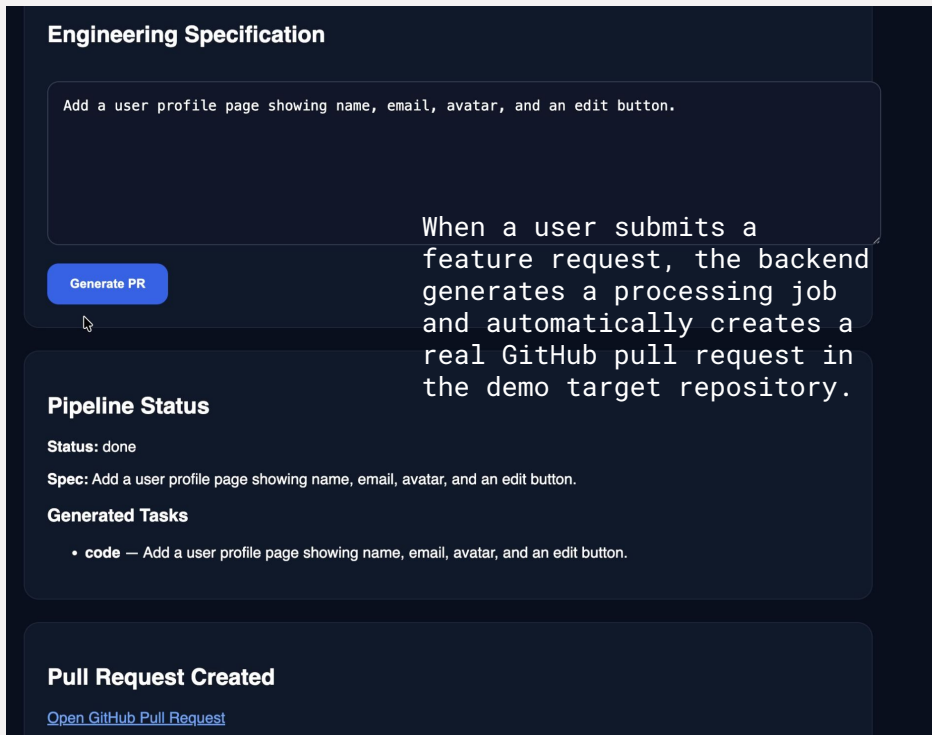
- feature name
- summary
- implementation tasks
- acceptance criteria
- affected files

The screenshot shows the IBM watsonx Orchestrate interface for the 'PR Description Agent'. The top navigation bar includes 'Agent chat / Manage agents / PR Description Agent'. The left sidebar has tabs for 'Profile', 'Knowledge' (selected), 'Toolset', 'Behavior', and 'Channels'. The main content area is divided into two sections: 'Knowledge' and 'Knowledge sources'. The 'Knowledge' section contains a description of the agent's capabilities, such as 'Relies on the models intrinsic ability to understand, plan and call tools and knowledge.' and 'Enables the model to think, act, observe and refine its approach until a task is completed.'. The 'Knowledge sources' section has a button to 'Add source'. On the right, there is a chat window titled 'Running in debug mode' showing a message from 'PR Description Agent' at 4:41 AM, which includes a JSON object with fields like 'title', 'feat', and 'body'.

Generates:

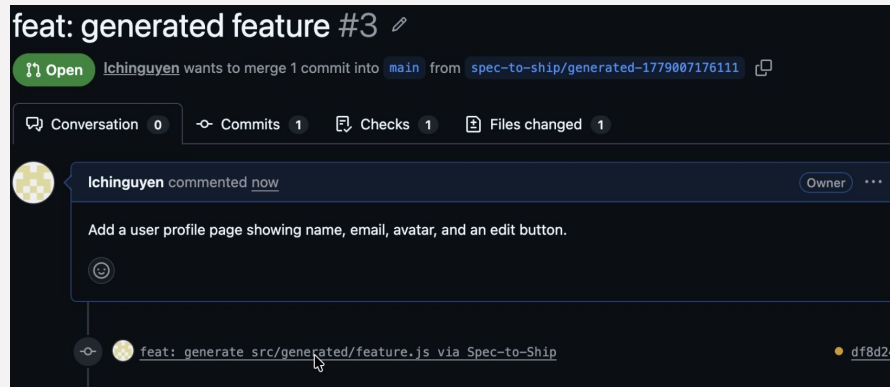
- professional PR titles
- PR summaries
- testing checklists

GitHub Automation



Spec-to-Ship uses the GitHub API through Octokit to:

- create a new branch
- commit generated files
- open a pull request in the demo target repository

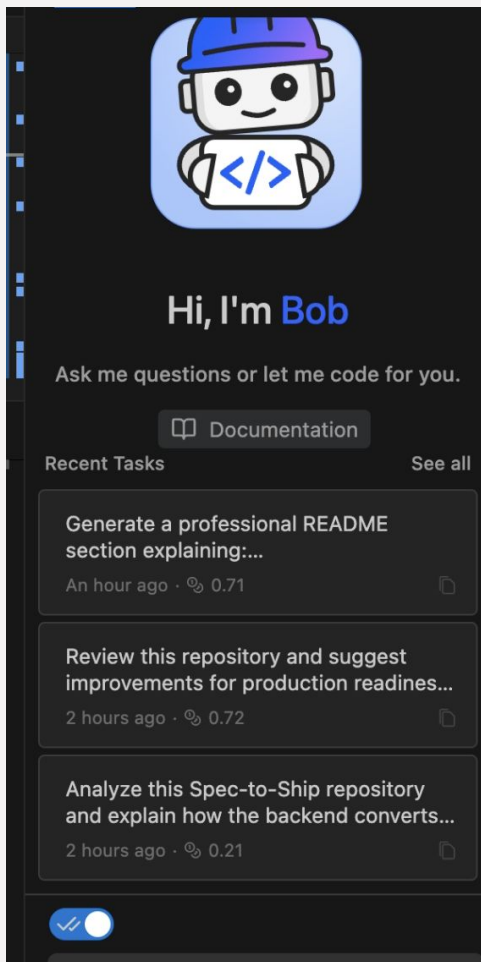


Testing repository: 'spec-to-ship-demo-target'

Screenshot:

- Generated GitHub PR
- Branch creation
- Changed files

Bob task session reports for judging included in: `bob_sessions/` folder



IBM Bob IDE usage:

Bob was used as the repository-aware development partner for this project.

- analyze the backend pipeline,
- explain how the system converts specs into pull requests,
- review the repository for production-readiness improvements,
- improve developer experience and demo clarity, generate README/documentation content.

Bob task session reports are included in:

```
bob_sessions/
```

Included Bob session artifacts:

```
bob_sessions/  
├─ bob-task-1-backend-pipeline-analysis.md  
├─ bob-task-2-production-readiness-improvements.md  
└─ bob-task-3-readme-generation.md
```